



2026-2027

SCHOOL OF COMPUTER APPLICATION

No SQL ASSIGNMENT:

Online food Ordering Records

Submitted_to:

Mr. Harendra Singh

Department: BCA(DS & AI

Date: 28th September

Submitted by:

Aditya Gond

Roll no:

(1250258036)

INTRODUCTION

The Online Food Delivery Management System is a MongoDB-based NoSQL project used to store and manage customer and order information. It contains details such as customer age, gender, occupation, monthly income, family size, customer type, and feedback.

In this project, different MongoDB operations like Insert, Find, Search, Sort, Count, Update, Delete, AND, and OR are performed on the customer data.

The main aim of this project is to understand how MongoDB can be used to manage real-world food delivery data efficiently.

1. CREATE DATABASE

MongoDB Query:

use foodOrderDB

Purpose: To create or switch to the food order database.

```
foodOrderDB> use foodOrderDB  
  
switched to db foodOrderDB
```

2. CREATE COLLECTION

MongoDB Query:

db.createCollection("Orders")

Purpose: Create a collection named Orders.

```
foodOrderDB> db.createCollection("Orders")  
  
{ ok: 1 }
```

3. INSERT ORDER DETAILS

MongoDB Query:

```
db.Orders.insertMany([
  {order_id: 1, item_name: "Paneer Butter Masala", ...},
  {order_id: 2, item_name: "Margherita Pizza", ...}
])
```

Purpose: Insert multiple order documents into the collection.

```
foodOrderDB> db.Orders.insertMany([
  {order_id: 1, item_name: "Paneer Butter Masala", ...},
  {order_id: 2, item_name: "Margherita Pizza", ...}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('65df...'),
    '1': ObjectId('65df...')
  }
}
```

4. Display Only Item Names

MongoDB Query:

```
db.Orders.find({}, {item_name: 1, _id: 0})
```

Purpose: To display only the names of the items ordered.

```
foodOrderDB> db.Orders.find({}, {item_name: 1, _id: 0})
[
  { item_name: 'Paneer Butter Masala' },
  { item_name: 'Margherita Pizza' }
]
```

5. Find Indian Category Orders

MongoDB Query:

db.Orders.find({category: "Indian"})

Purpose: To find all food orders categorized as Indian cuisine.

```
foodOrderDB> db.Orders.find({category: "Indian"})

[
  { order_id: 1, item_name: 'Paneer Butter Masala', category: 'Indian'... },
  { order_id: 3, item_name: 'Chicken Biryani', category: 'Indian'... }
]
```

6. Find Orders by Restaurant

MongoDB Query:

db.Orders.find({restaurant: "Domino's"})

Purpose: To find all orders placed at a specific restaurant.

```
foodOrderDB> db.Orders.find({restaurant: "Domino's"})

[
  { order_id: 2, item_name: 'Margherita Pizza', restaurant: 'Domino\'s'... }
]
```

7. Find Pending Orders

MongoDB Query:

db.Orders.find({status: "Pending"})

Purpose: To find all orders that are currently pending.

```
foodOrderDB> db.Orders.find({status: "Pending"})

[
  { order_id: 2, item_name: 'Margherita Pizza', status: 'Pending'... },
  { order_id: 8, item_name: 'Sushi Platter', status: 'Pending'... }
]
```

8. Find Orders with Rating Greater Than 4.5

MongoDB Query:

db.Orders.find({rating: {\$gt: 4.5}})

Purpose: To find highly rated food orders.

```
foodOrderDB> db.Orders.find({rating: {$gt: 4.5}})

[
  { order_id: 1, item_name: 'Paneer Butter Masala', rating: 4.8... },
  { order_id: 3, item_name: 'Chicken Biryani', rating: 4.9... }
]
```

9. Find Orders Priced Above ₹300

MongoDB Query:

db.Orders.find({price: {\$gt: 300}})

Purpose: To find premium or expensive food orders.

```
foodOrderDB> db.Orders.find({price: {$gt: 300}})

[
  { order_id: 2, item_name: 'Margherita Pizza', price: 400... },
  { order_id: 3, item_name: 'Chicken Biryani', price: 350... }
]
```

10. Find Orders Between ₹150 and ₹400

MongoDB Query:

db.Orders.find({price: {\$gte: 150, \$lte: 400}})

Purpose: To search for orders priced within a specific range.

```
foodOrderDB> db.Orders.find({price: {$gte: 150, $lte: 400}})

[
  { order_id: 1, item_name: 'Paneer Butter Masala', price: 250... },
  { order_id: 4, item_name: 'Hakka Noodles', price: 180... }
]
```

11. Find Orders with Quantity Greater Than 2

MongoDB Query:

db.Orders.find({quantity: {\$gt: 2}})

Purpose: To find bulk or large quantity orders.

```
foodOrderDB> db.Orders.find({quantity: {$gt: 2}})

[
  { order_id: 3, item_name: 'Chicken Biryani', quantity: 3... },
  { order_id: 5, item_name: 'Tacos', quantity: 4... }
]
```

12. Find Specific Item Details

MongoDB Query:

db.Orders.find({item_name: "Chicken Biryani"})

Purpose: To retrieve records for a specific food item.

```
foodOrderDB> db.Orders.find({item_name: "Chicken Biryani"})

[
  { order_id: 3, item_name: 'Chicken Biryani', category: 'Indian',
    price: 350, restaurant: 'Behrouz Biryani'... }
]
```

13. Find Delivered Indian Orders (AND Operation)

MongoDB Query:

db.Orders.find({category: "Indian", status: "Delivered"})

Purpose: To apply multiple conditions simultaneously.

```
foodOrderDB> db.Orders.find({category: "Indian", status: "Delivered"})

[
  { order_id: 1, item_name: 'Paneer Butter Masala', status: 'Delivered'... },
  { order_id: 3, item_name: 'Chicken Biryani', status: 'Delivered'... }
]
```

14. OR Operation (Dessert or Beverage)

MongoDB Query:

db.Orders.find({ \$or: [{category: "Dessert"}, {category: "Beverage"}] })

Purpose: To find orders that are either desserts or beverages.

```
foodOrderDB> db.Orders.find({ $or: [ {category: "Dessert"}, {category: "Beverage"} ] })

[
  { order_id: 6, item_name: 'Chocolate Brownie', category: 'Dessert'... },
  { order_id: 7, item_name: 'Cappuccino', category: 'Beverage'... }
]
```

15. Sort Orders by Price - Ascending

MongoDB Query:

db.Orders.find().sort({price: 1})

Purpose: To arrange orders from lowest price to highest price.

```
foodOrderDB> db.Orders.find().sort({price: 1})

[
  { order_id: 7, item_name: 'Cappuccino', price: 120... },
  { order_id: 6, item_name: 'Chocolate Brownie', price: 150... }
]
```

16. Sort Orders by Price - Descending

MongoDB Query:

db.Orders.find().sort({price: -1})

Purpose: To arrange orders from highest price to lowest price.

```
foodOrderDB> db.Orders.find().sort({price: -1})

[
  { order_id: 8, item_name: 'Sushi Platter', price: 600... },
  { order_id: 2, item_name: 'Margherita Pizza', price: 400... }
]
```

17. Find Top 3 Most Expensive Orders

MongoDB Query:

db.Orders.find().sort({price: -1}).limit(3)

Purpose: To display the 3 highest-priced orders.

```
foodOrderDB> db.Orders.find().sort({price: -1}).limit(3)

[
  { order_id: 8, item_name: 'Sushi Platter', price: 600... },
  { order_id: 2, item_name: 'Margherita Pizza', price: 400... },
  { order_id: 3, item_name: 'Chicken Biryani', price: 350... }
]
```

18. Update Order Status

MongoDB Query:

db.Orders.updateOne({order_id: 2}, {\$set: {status: "Delivered"}})

Purpose: To change the delivery status of a specific order.

```
foodOrderDB> db.Orders.updateOne({order_id: 2}, {$set: {status: "Delivered"}})

{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

MONGODB OPERATION USED:

Create Database: use foodOrderDB

Create Collection: createCollection()

Insert Operation: insertMany()

Find / Display: find()

Projection: Displaying specific fields only (item_name)

Filtering/Search: category, status, restaurant, etc.

Comparison Operators: \$gt, \$gte, \$lte

Multiple Conditions / AND: Using multiple conditions in one query

OR Operation: \$or

Sort Operation: sort() (Ascending: 1, Descending: -1)

Limit Operation: limit(3)

Update Operation: updateOne() + \$set

CONCLUSION

The Online Food Order Record Management System successfully demonstrates how a NoSQL database can be used to store and manage food delivery data. MongoDB makes it easy to add, search, update, and analyze order records. This project helped in understanding basic MongoDB concepts such as databases, collections, documents, queries, filtering, sorting, and projection.